A Übungsziel

Wir wollen grundlegende Konzepte des *Scripting* (Shell-Scripts) unter Linux kennenlernen, um wiederkehrende Vorgänge z.B. der Systemadministration zu automatisieren. Die Shell unter Linux/Unix (wir verwenden *bash*) ist rein textorientiert (im Gegensatz zur objektorientierten Powershell unter Windows, die wir demnächst untersuchen werden).

Lesetipps (auch als Unterlage für PLFs ohne Internet):

- https://goalkicker.com/ für Bash: "Stackoverflow" als PDF
- http://tldp.org/LDP/abs/abs-guide.pdf der ultimative Guide
- https://openbook.rheinwerk-verlag.de/shell_programmierung/index.htm auf deutsch, sehr umfangreich

B Plattform und Vorbereitung

Sie können ein Linux- oder Unix-System (real, virtuell) Ihrer Wahl verwenden. Sie benötigen nur einen gewöhnlichen User-Account[1] mit Zugang zur Kommandozeile *bash* (im Terminalfenster oder Textkonsole, auch per SSH oder WSL). Sie können jeden Editor ihrer Wahl verwenden. Notieren Sie alle verwendeten Kommandozeilen, fertige Skripts[2] und Beobachtungen in Ihrem Übungsprotokoll!

C Theorie und grundlegende Befehle

Die wichtigsten Befehle folgen als Übersicht. Details entnehmen Sie der Hilfe zum entsprechenden Befehl oder der Hilfe zu `bash(1)` (letzteres für die eingebaute Befehle).

C.1 Variablen

In Shellvariablen werden *immer* Zeichenketten gespeichert *(Unterschied zur Powershell!)*[3].

Zuweisung eines neuen Wertes:

```
name=wert
```

Achtung: Vor und nach dem "=" darf *kein* Leerzeichen sein!

```
name = wert   # Falsch --> keine Variablenzuweisung!
```

Hier arbeitet die Shell als Kommandointerpreter: Es wird das Programm `name` mit den zwei Parametern `=` und `wert` aufgerufen!

Wenn die Zeichenkette auch Leerzeichen enthält, müssen diese *maskiert* werden, im einfachsten Fall durch die Verwendung von doppelten Anführungszeichen:

```
name="zeichenkette mit leerzeichen"
```

Der Zugriff auf eine Variable erfolgt durch ein vorangestelltes $ Zeichen (die Shell erkennt das Metazeichen "$" und ersetzt dann den gesamten Variablenausdruck mit dem aktuellen Textinhalt der Variablen)

```
echo $name
```

bzw. auch bei Verwendung von doppelten Anführungszeichen:

```
echo "$name"
```

[1] eventuell müssen wir aber einige Tools installieren

[2] Zur besseren Lesbarkeit mit einer Schrift mit fixer Breite (`Schreibmaschinenschrift = monospace font`)! Am besten, Sie definieren dafür einfach eine *Formatvorlage*.

[3] Manchmal nutzt man jedoch auch die Möglichkeit, Variablen als `integer` bzw. als assoziatives Array (wie Java: `Map`) zu deklarieren (`declare` bzw. `typeset`)

Der Variablenname kann auch in `{ }` eingeschlossen werden *(um Mehrdeutigkeiten zu vermeiden)*:

```
s=sonder
echo ${s}zeichen
sonderzeichen
```

Achtung: in `''` schaut die Shell nicht hinein dh. Variablen werden nicht ausgewertet:

```
echo '$name'    # Ausgabe: $name
```

`${name:-word}` der Wert `word` wird verwendet, falls die Variable `name` nicht definiert oder leer ist

`${name:=word}` wie vorher, der Wert wird aber zusätzlich der Variable `name` zugewiesen.

Werden einem Skript Parameter übergeben, so sind sie als Spezialvariablen `$1`, `$2`, `$3`, … ansprechbar, der Name des Skripts (wie es aufgerufen wurde) steht übrigens in `$0`. Die *Anzahl* der übergebenen Kommandozeilenparameter steht in `$#`, und die weitere Spezialvariable `$@` beinhaltet ein Array mit allen Parametern.

C.2 Verzweigungen

Das Kommando `if` hat folgenden Aufbau:

```
if list
  then
    CMD1
  else
    CMD2
fi
```

Der `else`-Zweig kann entfallen. `list` ist dabei ein beliebiges Kommando (häufig `test` oder die Abkürzung `[ ... ]`) bzw. eine Pipe oder mehrere Kommandos in Klammern – betrachtet wird stets der *Exit-Status* (`0` = True, alles andere = False). Mehrere Bedingungen kann man mit `-a` (*und*) bzw. `-o` (*oder*) verknüpfen.

Beispiel:

```
if [ "$eat" = apfel -o "$eat" = birne ]      # oder: if test "$eat" = apfel -o "$eat" = birne
then
   echo Ich liebe Äpfel und Birnen!
fi
```

Wichtig:

- `"$eat"` ein Wort
- `$eat` vielleicht viele Wörter (falls Leerzeichen in `$eat`)

Das Kommando `CMD1` (oder beliebig viele weitere Befehlszeilen) nach `then` wird ausgeführt wenn das Kommando nach `if` erfolgreich war, also den *Exit-Status* `0` zurück liefert.

Statt `if` kann man auch folgende (Kurz-)Schreibweisen verwenden:

`CMD1 && CMD2`	`CMD2` wird ausgeführt wenn `CMD1` erfolgreich war
`CMD1 || CMD2`	`CMD2` wird ausgeführt wenn `CMD1` nicht erfolgreich war

C.3 Schleifen

C.3.1 while-Schleife

Führe `COMMANDS` aus solange `TEST` erfolgreich ist:

```
while TEST
do
  COMMANDS
done
```

Beispiel:

```
i=10
while [ $i -gt 0 ]      # Wichtig: Leerzeichen!
do
    echo $i
    let i=i-1
done
```

(Mit `let` *kann man die Shell anweisen, arithmetische Ausdrücke auszuwerten, also zu rechnen)*

C.3.2 for-Befehl

Führe die `COMMANDS` für jeden Wert in `WORDS` aus (iteriert über jedes Wort). Die Variable `NAME` nimmt dabei den aktuellen Wert an (siehe etwa Beispiel bei D.2):

```
for NAME in WORDS
do
  COMMANDS
done
```

Beispiel:

```
for var in Osten Süden Westen
  do
    echo Die Sonne steht im $var
  done
```

Wird `WORDS` nicht angegeben so wird `$@` (= alle Parameter des Skripts, also `$1 $2 $3` …) verwendet.

Beispiel *(bitte in einem Mini-Skript ausprobieren!)*:

```
for par
do
  echo Kommandozeilenparameter ist: $par
done
```

Beispiel: Dateien in Verzeichnis abarbeiten

```
for datei in $(ls *.txt)    # Kommandozeilensubstitution $(...) siehe nächster Abschnitt!
do
  echo Textdatei gefunden: $datei
done
```

Tipp: Zählschleife mit `seq` oder `for ((i=1;i<=10;i++))`

C.4 Nützliche Befehle für Scripts und Pipes

`expr, eval, $(( )), let`	Rechnen in der Shell
`$( … )`	Textausgabe der Befehlskette in Klammern als Text substituieren, z.B.: `echo hallo > $(date +%Y%m%d).log` Alternativ gibt es eine Variente mit *Backticks*, die schlechter lesbar ist und vermieden werden sollte: `echo hallo > `date +%Y%m%d`.log`
`cut`	*Ausschneiden* von *Spalten* der Eingabe
`head, tail`	Erste bzw. letzte Zeilen einer Datei ausgeben
`sort`	Sortieren
`uniq`	Zusammenfassen gleicher Zeilen der Eingabe
`grep`	Suche nach Suchmustern
`wc`	Zählt Zeichen, Wörter (“Word Count”), Zeilen….
`basename, dirname`	Verzeichnis und Suffix von Dateinamen verarbeiten Alternativ: ${var// … / … } oder $(var#…}

`read`	Einlesen eines/mehrerer Werte von der Standardeingabe in Variable, z.B.:
	`read a b c`
	`echo erster Parameter ist $a, zweiter ist $b, Rest ist in $c`

`awk`	Muster in der Eingabe erkennen und verarbeiten (rechnen), kann auch zum Verarbeiten von “Spalten” in der Eingabe verwendet werden, z.B. `echo 2 + 3 | awk { erg=$1+$3; print $erg }`
`sed`	Streameditor zum Filtern und Umwandeln von Text, z.B. (search/replace): `echo 15% | sed s/%//g`
`exit`	Beenden (mit Rückgabewert), z.B. `exit 0` bei Erfolg, `exit 1` bei Fehler. Den Wert des Exit-Status eines Scripts oder Programms kann man in einem Script abfragen (mit `test`!)

C.5 Ausführen von Shell-Scripts

Damit ein Script “direkt” ausführbar ist

- muss das `x`-Bit (Execute) bei den Dateieigenschaften gesetzt sein,
- sollte in der ersten Zeile “`#!`” und das auszuführende Programm stehen *(best practice)*: **#!/bin/bash**
- die Datei im Suchpfad (`PATH`) gefunden werden (das aktuelle Verzeichnis ist i.d.R. nicht im Pfad). Deshalb muss das Skript entweder in einen Ordner des Suchpfads kopiert werden (z.B. `/usr/bin` oder `~/bin`) oder mit einer absoluten oder relativen Pfadangabe aufgerufen werden also i.d.R. `./myscript`, wenn das Script im aktuellen Ordner liegt.
- Alternative (vermeiden): `bash pfad/zum/script`

  Achtung: `sh pfad/zum/script` funktioniert *manchmal* auch, aber `sh` ist nicht so mächtig wie `bash`.

Fehlersuche:

- Mit “`set -x`” bekommt man zusätzliche Ausgaben *(hilfreich für’s Debugging)*!
- Mit “`set -e`” wird das Script beim ersten Fehler (exit-code nicht 0) abgebrochen

C.6 Ein einfaches Skript-Beispiel

```
#!/bin/bash
# Kommentar
echo Parameter1: ${1:-leer} # Gibt $1 aus, falls gesetzt, sonst "leer"
LOGFILE=$(date +%Y%m).log
if [ -e "$1" ]              # oder:   if test -e "$1"
then
  echo Datei $1 existiert > $LOGFILE
  cp $1 $1.bak
fi
exit 0
```

D Endlich: Die Aufgaben

D.1 Aufgabe: Der Launen-Simulator

Schreiben Sie ein Bash-Script namens `laune`, das die aktuelle "Laune" des Computers abfragt und darauf reagiert.

D.1.1 Ziel der Übung

- Kommandozeilenargumente auslesen
- Eingaben wiederholt einfordern
- Nur bestimmte gültige Antworten akzeptieren
- Mit Exit-Codes arbeiten
- Einfache Kontrollstrukturen und Schleifen nutzen

D.1.2 Aufgabenstellung

Ihr Script `laune` soll folgende Funktion erfüllen:

1. **Der Benutzer übergibt einen Satz** auf der Kommandozeile, z.B.

   ```
   ./laune "Wie ist deine Laune?"
   ```

2. Das Script gibt diesen Satz aus und fordert den Benutzer auf, die Laune des Computers einzugeben. **Gültige Antworten sind:**

   - `gut` → Exit-Code **0**
   - `schlecht` → Exit-Code **1**
   - `müde` → Exit-Code **2**

3. **Alle anderen Eingaben** sollen mit einer leicht genervten Computer-Antwort quittiert werden, z.B.: "Ich bin ein Computer, kein Gedankenleser!" Danach wird die Frage **wiederholt**, bis eine gültige Antwort kommt.

4. Das Script endet mit dem passenden Exit-Code.

D.1.3 Beispielablauf

```
$ ./laune "Wie ist deine Laune?"
Wie ist deine Laune?
fantastisch
Was soll das heißen?
Wie ist deine Laune?
müde
$ echo $?
2
```

D.1.4 Testscript `launetest`

Erstellen Sie anschließend ein Testscript `launetest`, das Ihr Script mehrfach aufruft:

```
#!/bin/bash
while true
do
    if ./laune "Wie fühlst du dich heute? (gut/schlecht/müde)"
    then
        echo "Der Computer ist gut drauf!"
    else
        status=$?
        if [ $status -eq 1 ]; then
            echo "Der Computer ist schlecht drauf!"
        elif [ $status -eq 2 ]; then
            echo "Der Computer ist müde!"
        fi
        echo "Test beendet."
        exit 0
    fi
done
```

Kommentieren Sie jede Zeie des Testscripts und beschreiben Sie, was hier passiert! Rufen Sie nun `launetest` auf und geben Sie ein paar Beispielantworten ein. Machen Sie von Ihrem Dialog mit dem Computer einen Screenshot und fügen Sie diesen in Ihr Protokoll ein!

D.2 Aufgabe: Alte Dateien entsorgen

Ein Script `zapold` *(zap old files)* soll die *N* ältesten Dateien in einem Verzeichnis (z.B. dem aktuellen Ordner) *löschen* (*N* ist ein Parameter mit dem das Skript aufgerufen wird). Um die drei ältesten Dateien zu löschen, gibt man also ein: `zapold 3`.

Tipp 1 zum Ansatz: Sortieren Sie die Ausgabe eines Verzeichnis-Listings nach dem Datum und und filtern Sie auf die ersten bzw. letzten `N` Dateinamen. Diese Liste können Sie dann mit `for DATEI in ...` verarbeiten.

Tipp 2: Für Testdaten: Der Befehl

```
touch -d "2 day ago" x
```

erzeugt eine 2 Tage "alte" Datei namens `x`. Nutzen Sie das, um einen *ganzen Haufen* Dateien zum Testen zu erzeugen, z.B. mit[4]

```
for i in 1 2 3 4 5 6 7 8 9 10
do
```

[4] echte Hacker verwenden hier `seq` oder `for i in {1..10}`

```
    touch -d "$i day ago" file${i}daysold.txt
done
```

Tipp 3: das Script und die Testdaten sollten *nicht* im selben Ordner liegen, sonst löscht man irrtümlich das Script…

Tipp 4: Zuerst sollte man im Script die Löschbefehle mit

```
echo rm ...$x...
```

nur ausgeben und kontrollieren. Man kann dann das `echo` entfernen oder die Ausgabe in einer neuen Shell ausführen:

```
zapold 3 | bash
```

D.3 Aufgabe: Thumbnails

D.3.1 Verkleinern von Bilddateien

Im Web braucht man Fotos mit niedriger Auflösung (*Thumbnails*). Ihr Script soll ein JPG-Bild auf "240x100" oder kleiner verkleinern (Dateiname wird dem Script `thumbnail` als Parameter übergeben). *Testen* Sie das Script, indem Sie z.B. die Bilder der Katzen Lola und Nala verwenden!

- *Tipp:* Kommando `convert(1)` – ein Teil des Pakets `imagemagick` (ggf. installieren!)
- Wie kann man einen neuen Dateinamen (z.B. `lola_TN.jpg` aus `lola.jpg`) mit der "richtigen" Endung für das Ergebnisbild erzeugen? Hinweis: Es reicht, wenn Ihr Script vorerst nur Dateien mit der Endung `.jpg` bearbeitet!

D.3.2 Bonus: Thumbnails generieren

Erweiterung: Das Script soll für *alle* Bilder (z.B. JPGs) in einem Ordner *Thumbnails* erzeugen!

D.4 Aufgabe: Spooler

D.4.1 Grundfunktion

Sie benötigen zur Überwachung/Wartung einer Anwendung einen einfachen *Spooler*, also ein Shell-Script `spooler`, das den Inhalt eines Ordners `~/spool/queue` periodisch auf Dateien überwacht. Findet es in dem Ordner Dateien, so fügt es an jede gefundene Datei eine Dateiendung `.JJMMTT-hh:mm:ss` an (= aktuelle Datum/Uhrzeit), verschiebt sie sodann in den Ordner `~/spool/queue/.history` und komprimiert sie mit `gzip(1)`. Nach einem solchen Vorgang sollte der `queue` -Ordner "geleert" sein (es sei denn, neue Dateien sind in der Zwischenzeit eingetroffen). Nach 1 Sek. beginnt der `spooler` erneut einen Scan (= Listing) des Ordners und führt wieder Umbenennung/Verschieben/Komprimieren aus (Endlosschleife).

Zum Testen verwenden Sie folgendes Script `producer`, das in halbwegs zufälligen Abständen (zwischen 0 und 4 Sekunden) Dateien im Ordner `~/spool/queue` mit verschiedenem Inhalt erzeugt. Sie können das z.B. in einem anderen Terminal starten:

```
#!/bin/bash
DIR=~/spool/queue
cd $DIR
while true
do
  FILENAME=log$RANDOM
  SLEEP=$(($RANDOM % 5))
  echo Content for file $FILENAME: Sleep: $SLEEP seconds  | tee $FILENAME
  echo $0: Sleeping for $SLEEP seconds...
  sleep $SLEEP
done
```

Hinweis: `tee` liest einfach alles von der Standardeingabe und schreibt es auf die Standardausgabe, schreibt die Daten aber gleichzeitig in einer Datei (Parameter) mit.

Sobald die obige Grundfunktion funktioniert (testen!), erweitern Sie Ihren Spooler wie folgt:

D.4.2 Protokolldatei – Spooler mit Logging

Ihre Lösung soll auch jedes Mal, wenn eine Datei gefunden, komprimiert und verschoben worden ist, eine entsprechende Zeile (mit *Tag* – z.B. "`spooler:`", dann folgt der Name der Datei) in einem *Log-File* `~/spool/log/spooler.log` mitprotokollieren (also einfach eine neue Zeile daran anhängen).

D.4.3 Warnmeldung – Alle guten Dinge sind drei

Sollte in einer Datei, die von `spooler` komprimiert und verschoben wird, die markante Zeichenkette "`4 seconds`" vorkommen, soll in der Logdatei (zusätzlich) eine Zeile mit einer Warnung geschrieben werden!

D.4.4 Bonus: Syslog

Für Linux/Unix-Gurus: Schreiben Sie die Log-Information nicht in `~/spool/log/spooler.log`, sondern in den *System-Log* (Tipp: `logger(1)`)

D.5 Bonusaufgabe: Freier Plattenplatz

Eine wichtige Tätigkeit beim Überwachen eines Servers ist der freie Plattenplatz. Schreiben Sie ein Skript das den freien Speicherplatz überwacht und eine Meldung beim Überschreiten eines Grenzwerts ausgibt.

- Welches Kommando zeigt den freien Plattenplatz (*disk free*)?
- Welches Kommando kann mit Teilen der Eingabezeile "rechnen"?
- Geheimnisvoller *Tipp:* "`$5 + 0`" macht aus einem Text eine Zahl

D.6 Bonusaufgabe: `rename.sh`

- Zum Testen erstellen Sie in einem Unterordner:
  - 20 Dateien namens `file1.text` bis `file20.text` an,
  - sowie 10 Dateien `file1.log` … `file10.log`
- Schreiben Sie das Script `rename.sh` mit den beiden Parametern `oldext` und `newext`, das die Dateien umbenennt.
  - Beispielaufruf: "`bash rename.sh .text .txt`" macht im Arbeitsverzeichnis aus `file*.text` → `file*.txt`.
- Das Script soll als Parameter den Pfadnamen des Unterordners annehmen (Vorgabe, wenn ohne Parameter: "`.`").
  - Beispielaufruf: "`bash rename.sh .text .txt ../texte/myfliles`" sucht im angegebenen Ordner.
- Das Script soll bei jeder Datei fragen, ob man sie wirklich umbenennen möchte. Dazu gibt es den optionalen Parameter `-a` ("ask").
  - `bash rename.sh -a .text .txt /temp/myfliles` fragt bei jeder Datei nach: [j/n].
  - `bash rename.sh .text .txt /temp/myfliles` fragt nicht nach

Viel Spaß!